Lightweight Anomaly Detection for IoT Edge: A Contamination Sensitivity Analysis of Isolation Forest and Robust Random Cut Forest

Zarish Fatema, Student
Muhammad Abdullah, Student
Taimur Tariq Sethi, Student
PNEC, NUST, KHI Dec 2025

The proliferation of Internet of Things (IoT) devices across critical infrastructure has created a vast, vulnerable attack surface where traditional, centralized Deep Learning (DL) solutions are rendered ineffective by unacceptable latency and device resource constraints. To address the acute requirement for real-time edge security, this research benchmarks two prominent lightweight, unsupervised anomaly detection algorithms; Isolation Forest (IF) and Robust Random Cut Forest (RRCF), using network flow traffic derived from the highly realistic CIC IoT-DIAD 2024 dataset. The study specifically quantifies the trade-off between detection robustness (F1 Score stability) and Operational Efficiency (Evaluation Time) across three critical contamination thresholds (0.01, 0.05, and 0.10). Results demonstrated a sharp dichotomy; while IF was computationally superior (0.306s average latency), it proved statistically ineffective ($F1 \approx 0.069$). Conversely, RRCF achieved markedly superior detection robustness ($F1 \approx 0.394$) maintaining high stability against varying threat levels. Though RRCF exhibited a 4.8 x higher latency in evaluation (≈1.482 second), it is concluded to be the optimal operational choice, confirming that maximizing the critical F1 score stability, rather than minimizing latency in isolation, is the decisive factor for implementing viable and effective lightweight Network Intrusion Detection Systems at the IoT edge.

1. INTRODUCTION

The proliferation of Internet of Things (IoT) devices across critical infrastructure has created a vast, vulnerable attack surface. Many IoT devices suffer from intrinsic vulnerabilities, including weak authentication protocols, unpatched firmware, and reliance on insecure data transmission methods, all of which hackers can exploit to manipulate systems or steal sensitive data [1]. A particularly critical risk involves the manipulation of sensor data, which can lead to inaccurate readings in crucial systems like industrial control processes or patient monitoring, resulting in safety hazards, operational downtime, and financial loss [1].

The inherent resource limitations of IoT devices namely low processing power, minimal memory, and constrained bandwidth; render traditional, centralized Network Intrusion Detection Systems (NIDS) ineffective or entirely inapplicable [4]. Traditional Intrusion Detection Systems (IDSs) typically fall into two categories: signature-based, which rely on databases of known intrusions, and anomaly-based, which learn normal system behavior [12].

Existing IDS systems based on Deep Learning (DL) approaches, utilizing models like Recurrent Neural Networks (RNN) and Bidirectional Long Short-Term Memory (Bi-LSTM), have demonstrated exceptional statistical performance, often achieving high detection rates and F1-measures exceeding 99% in controlled environments using datasets like BoT-IoT [3]. The training and inference requirements for DL models, coupled with the massive data volumes generated by IoT devices, create substantial computational demands and transmission overheads. This challenge highlights a critical dichotomy; while centralized DL solutions achieve high statistical accuracy, their latency and resource consumption negate their operational feasibility in a resource-constrained, real-time IoT setting. For mission-critical IoT applications, such as those governing utility grids or hospital systems, immediate threat detection and intervention are mandatory [8].

Therefore, the necessity arises for highly efficient, lightweight anomaly detection models capable of executing directly on edge devices [8].

Anomaly detection is the cornerstone of effective IoT security monitoring, enabling the early identification of security breaches, equipment malfunctions, and subtle cyber-attacks by flagging significant deviations from the established normal operational profile [10]. Tree-based ensemble methods, such as Isolation Forest (IF) and Robust Random Cut Forest (RRCF), are attractive candidates for edge deployment due to their computational efficiency and minimal memory footprint [8].

This research is dedicated to comparing the suitability of two prominent anomaly detection algorithms; Isolation Forest (IF) and Robust Random Cut Forest (RRCF), as components of a lightweight NIDS operating on IoT network flow data. The research utilizes network flow data derived from the CIC IoT-DIAD 2024 dataset, a state-of-the-art collection generated for both device identification and anomaly detection, featuring network traffic from 105 IoT devices under benign and malicious conditions. The experimental framework employs a semi-supervised learning approach, training models solely on benign network flows and evaluating their efficacy against known attack traffic. The analysis systematically investigates the crucial impact of the contamination hyperparameter across three distinct levels (0.01, 0.05, and 0.10) [9]. The study specifically quantifies the trade-off between detection robustness (F1 Score stability) and Operational Efficiency (Evaluation Time) across three critical contamination thresholds (0.01, 0.05, and 0.10). Results demonstrated a sharp dichotomy: while IF was computationally superior (0.306s average latency), it proved statistically ineffective ($F1 \approx 0.069$). Conversely, RRCF achieved markedly superior detection robustness ($F1 \approx 0.394$) maintaining high stability against varying threat levels. Though RRCF exhibited a $4.8 \times$ higher latency in evaluation (≈ 1.482 second), it is concluded to be the optimal operational choice, confirming that maximizing the critical F1 score stability, rather than minimizing latency in isolation, is the decisive factor for implementing viable and effective lightweight Network Intrusion Detection Systems at the IoT edge.

The outcome of the study contributes to the field of intrusion detection by quantifying the trade-off between statistical detection metrics (Precision, Recall, F1 Score) and operational efficiency (Evaluation Time) for IF and RRCF in a flow-based IoT NIDS context, providing a detailed comparative analysis of the sensitivity of both algorithms performance to varied, fixed contamination thresholds [3] and establishing a criterion for selecting superior IDS algorithms based on practical resource constraints, moving beyond pure statistical accuracy to prioritize low latency and operational viability for resource-limited edge computing environments.

2. METHODOLOGY

2.1. Isolation Forest (IF) Mechanics and Formulation

The methodology employed in this research focuses on a semi-supervised anomaly detection framework comparing the performance characteristics of Isolation Forest and Robust Random Cut Forest when applied to IoT network flow features. The models are exclusively trained on data labeled as "Benign" [9]. This approach simulates real-world scenario requirements where system behavior during normal operation is well-defined, but malicious traffic patterns ("Intrusion") are unknown or too varied to be exhaustively labeled a priori. The experimental input utilizes network flow traffic data derived from the CIC IoT-DIAD 2024 dataset, a highly realistic collection of traffic generated in an extensive IoT

topology using 105 real devices from multiple vendors. The dataset encompasses traffic from 33 distinct cyberattacks categorized across seven classes, including DDoS, DoS, Reconnaissance, Web-based, Brute Force, Spoofing, and Mirai. The models were trained and evaluated on 10 numerical flow features critical for analyzing network behavior: 'Flow Duration', 'Total Fwd Packet', 'Total Bwd packets', 'Total Length of Fwd Packet', 'Total Length of Bwd Packet', 'Fwd Packet Length Max', 'Bwd Packet Length Max', 'Fwd Header Length', 'Bwd Header Length', and 'Packet Length Std' [9]. Prior to training, all numerical features are standardized through scaling using a MinMaxScaler [9]. Data cleaning involves replacing infinite or excessively large values with not-a-number (NaN) and subsequently dropping rows containing NaN values to ensure dataset integrity [11]. This preprocessing step is vital for ensuring that outlying values, which might otherwise skew distance-based anomaly detection methods, are handled appropriately, preserving the fidelity of the subsequent anomaly identification process [9]. The reliance of both models on the contamination parameter highlights a critical dependency in unsupervised anomaly detection. As noted in existing literature, the final anomaly score and subsequent classification are highly dependent on having a reasonably accurate initial estimate of the anomaly rate (C) [4]. By testing three distinct contamination values, 1%, 5%, and 10%, the research explores the practical limitation of model deployment. If the actual rate of attack traffic in the environment deviates significantly from the predetermined C used to set the threshold, model performance, particularly the F1 score, can be heavily compromised [4]. The comparative analysis of IF and RRCF across these varying assumed outlier conditions will quantify which algorithm exhibits greater robustness and F1 stability in the face of fluctuating or unknown threat levels, providing essential information for operational risk assessment.

2.2. Mathematical Formulation of Isolation Forest Anomaly Score

The Isolation Forest anomaly score $s(x, n)$ for an observation x, given n total observations, is calculated using the expected path length $E[h(x)]$ across the ensemble of isolation trees:

$$s(x, n) = 2^{-\frac{E[h(x)]}{c(n)}} \tag{1}$$

Where:

- $E[h(x)]$ represents the average path length (number of edges/partitions) from the root node to the terminal node for observation x across all Isolation Trees in the ensemble [2].
- $c(n)$ is a normalization constant used to normalize the path length. It represents the average path length of unsuccessful searches in a Binary Search Tree (BST) of n observations.
- The normalization constant $c(n)$ is formally defined as:

$$c(n) = 2H(n-1) - \frac{2(n-1)}{n} \tag{2}$$

where $H(i)$ is the harmonic number [2].

A computed anomaly score $s(x, n)$ that approaches 1 indicates an anomaly (short path length). Scores approaching 0.5 or 0 indicate a normal point.

2.3. Robust Random Cut Forest (RRCF) Mechanics and Formulation

Robust Random Cut Forest (RRCF) is an ensemble method designed specifically for detecting outliers in dynamic data streams, offering robustness against high-dimensional data, noise, and the masking effect of duplicates and near-

duplicates. This resilience is particularly valuable in network telemetry, which is often characterized by correlated numerical attributes. The RRCF anomaly score is based on the Co-Displacement metric, which quantifies the structural influence of a data point on the remainder of the tree. Points identified as non-parametric anomalies impose a larger "externality" on the forest structure, leading to higher Co-Displacement scores [11].

2.4. Mathematical Formulation of Robust Random Cut Forest (RRCF) Co-Displacement

RRCF extends the standard concept of displacement, which measures the expected change in the total complexity of a tree when a point is removed. The model complexity of a tree T, $|M(T)|$, is initially defined as the sum of path lengths for all points y in the training dataset Z :

$$|M(T)| = \sum_{y \in Z} f(y, Z, T) \tag{3}$$

where $f(y, Z, T)$ is the depth of y in tree T.

To address the issue of outlier masking caused by duplicates, RRCF utilizes the Collusive Displacement (CoDisp). This metric measures the expected structural change if a set of "colluders" C (which includes x) were simultaneously removed, thus ensuring that the anomaly signal is not diluted by near-duplicates [3]. The Co-Displacement is defined as:

$$CoDisp(x, Z, T) = \sum \ (f(y, Z, T) - f(y, Z - \{x\}, T')) \tag{4}$$

The implementation utilizes a custom scoring function (get_rrcf_anomaly_scores) which builds 50 trees (N_{TREES}) with a fixed sample_size of 256 points for each tree. Unlike the native scikit-learn IF implementation, the RRCF threshold is determined manually based on the contamination rate C. The threshold is established by calculating the $1 - C\ x\ 100^{th}$ percentile of the RRCF scores derived exclusively from the benign training data [9]. Points in the evaluation phase with scores exceeding this threshold are classified as intrusions. This custom thresholding mechanism ensures a fair and direct comparison of model performance sensitivity relative to the IF model under identical contamination assumptions.

3. METHODOLOGY

The evaluation focuses on two core dimensions critical for IoT deployment: detection performance and operational efficiency. Models were trained on the combined benign dataset and subsequently evaluated against 16 individual attack files, including various DDoS, flood, brute force, and injection attacks [9]. The performance metrics used are Precision, Recall, F1 Score, and Accuracy, calculated against the true labels of the attack files, with 'Intrusion' designated as the positive class [1]. In the context of anomaly detection, which involves significant class imbalance, the F1 Score is the most reliable measure of overall model robustness, as accuracy alone can be misleading. The statistical analysis focuses on the influence of the three tested contamination values (C=0.01, C=0.05, and C=0.10) on the aggregated classification metrics across all 16 attack files [9]. The Precision score remained consistently high (1.0) across all runs for both algorithms, indicating that when a point was flagged as an anomaly, it was almost always a true intrusion. Therefore, the F1 Score, which balances Recall and Precision, serves as the primary indicator of overall effectiveness.

3.1. Isolation Forest (IF) Detection Performance

Isolation Forest (IF) consistently exhibited poor detection capability (Recall and F1 Score) across all contamination levels. The highest average F1 score achieved was 0.069 at $C = 0.01$ summarized in Table 1 & depicted in Figure 1. This

suggests that for the tested network flow dataset and feature set, IF struggled significantly to isolate intrusion points from the dense clusters of normal traffic.

Table 1: Comparative Performance Metrics (Averages Across All Attack Files)

Contamination (C)	Algorithm	Avg. Precision	Avg. Recall	Avg. F1 Score	Avg. Accuracy
0.01	Isolation Forest	1.000	0.002	0.005	0.002
0.01	RRCF	1.000	0.117	0.186	0.117
0.05	Isolation Forest	1.000	0.037	0.049	0.037
0.05	RRCF	1.000	0.219	0.375	0.219
0.10	Isolation Forest	1.000	0.052	0.069	0.052
0.10	RRCF	1.000	0.231	0.394	0.231

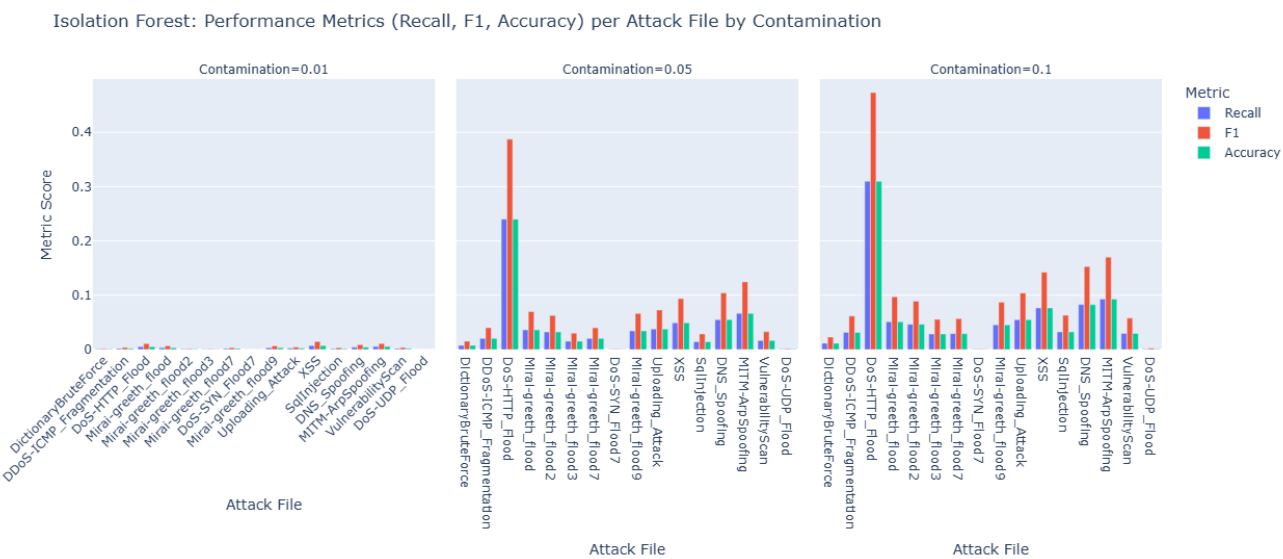

Figure 1: Isolation Forest: Performance Metrics (Recall, F1, Accuracy) per Attack File by Contamination

3.2. Isolation Forest (IF) Operational Efficiency

Crucially, Operational Efficiency is quantified by measuring the Evaluation Time (in seconds) required for each model to classify data points within an attack file. This metric directly determines the model's suitability for real-time edge processing, where latency must be minimized [8]. The average time required to evaluate the models against all 16 attack files is summarized in Table 2 and depicted in Figure 2 and Figure 3 respectively.

Table 2: Operational Efficiency Comparison (Average Evaluation Time)

Algorithm	Average Evaluation Time (s)	Relative Efficiency
Isolation Forest (IF)	0.306	4.8 x Faster
Robust Random Cut Forest (RRCF)	1.482	Baseline Latency

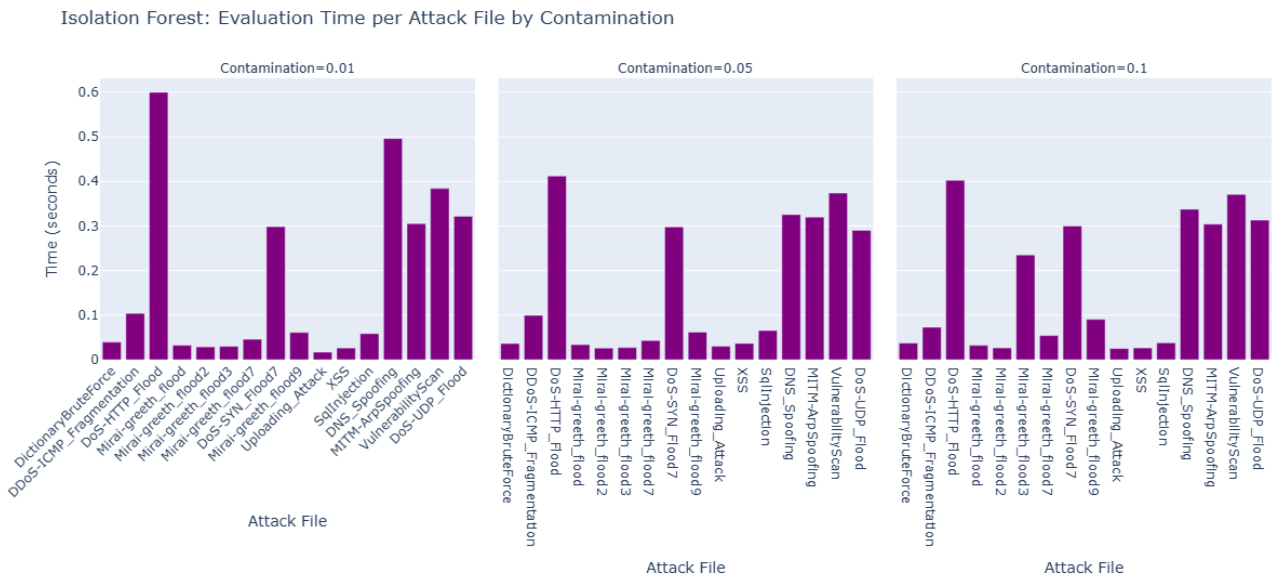

Figure 2: Isolation Forest: Evaluation Time per Attack File by Contamination

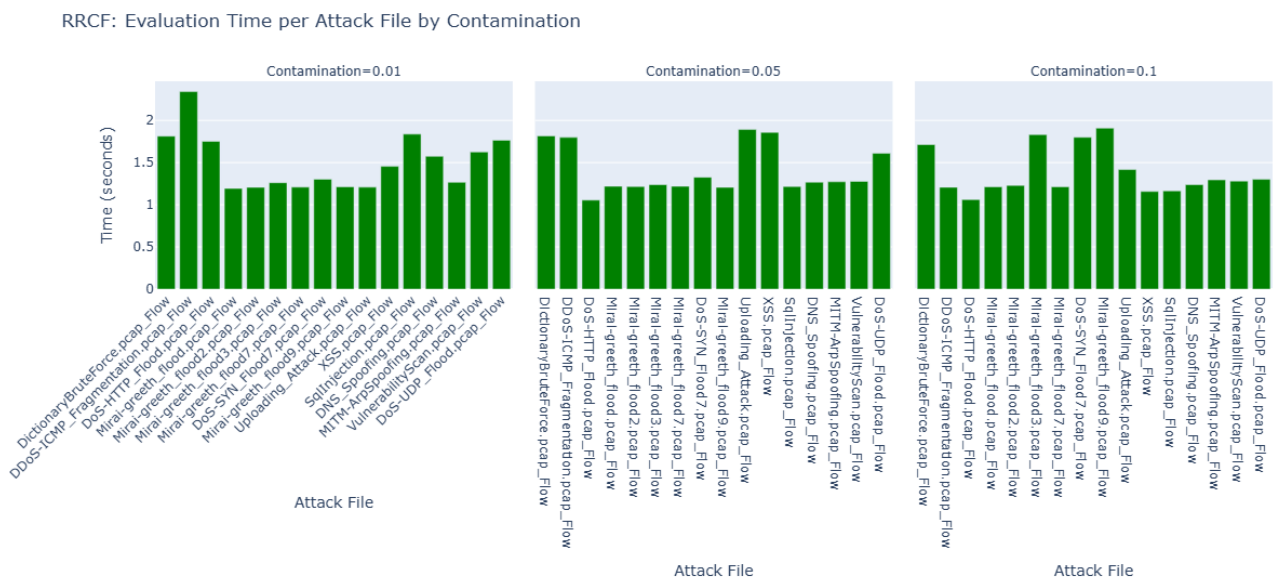

Figure 3: RRCF: Evaluation Time per Attack File by Contamination

3.3. Robust Random Cut Forest (RRCF) Detection Performance and Stability

Robust Random Cut Forest (RRCF) demonstrated markedly superior performance as depicted in Figure 4. At the lowest assumed contamination (C=0.01), RRCF achieved an average F1 Score of 0.186, rising substantially to approximately 0.394 at contamination levels C=0.05 and C=0.10. This indicates that RRCF is significantly more effective at identifying and separating anomalies in this specific network flow environment. Crucially, the F1 score stability for RRCF was

excellent between C=0.05 (0.375) and C=0.10 (0.394), suggesting its detection effectiveness is maintained even if the true anomaly rate fluctuates around these higher thresholds.

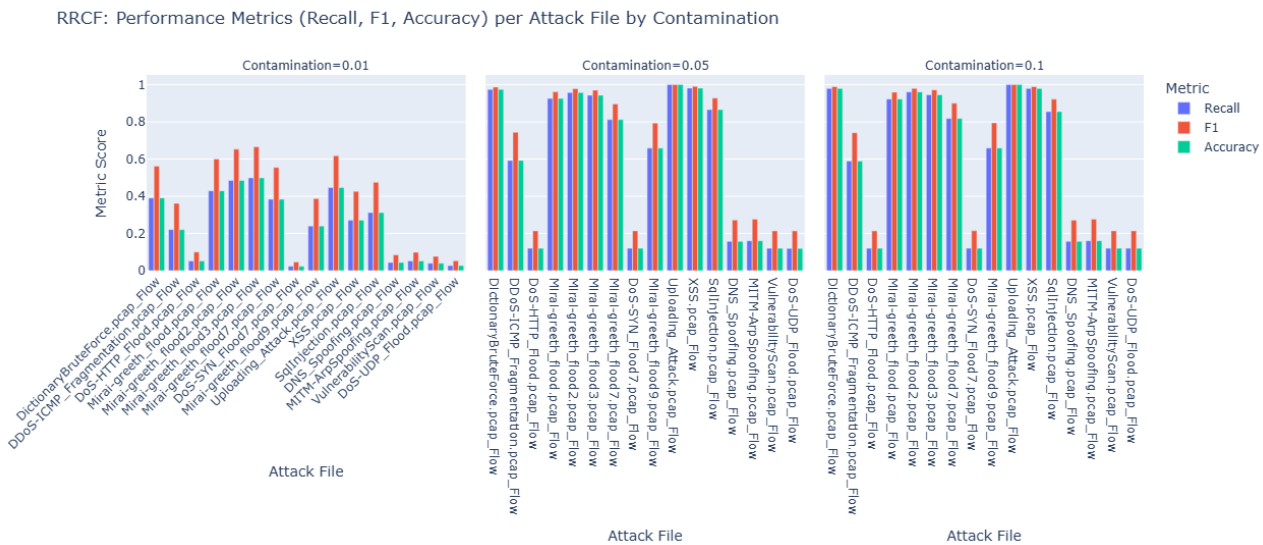

Figure 4: RRCF: Performance Metrics (Recall, F1, Accuracy) per Attack File by Contamination

3.4. Robust Random Cut Forest (RRCF) Operational Efficiency

RRCF's slower evaluation speed, as depicted in Table 2, is attributed to the computational overhead of its specific scoring mechanism (Co-Displacement) and its implementation structure, which is designed more for streaming data characteristics rather than raw prediction throughput in this batch-evaluation environment [16].

4. DISCUSSION

The empirical results highlight a sharp trade-off: RRCF delivers robust detection performance ($F1 \approx 0.394$) but at a high operational cost (1.482 s average latency), while IF is extremely efficient ($Latency \approx 0.306s$) but sacrifices almost all detection capability ($F1 \approx 0.069$). Given the need for effective anomaly detection in critical infrastructure, where failing to detect an attack is a severe risk [1], the performance of IF is deemed unacceptable, despite its speed advantage. The structural robustness of RRCF, evidenced by its superior F1 score $\approx 5.7 \times$ higher than IF's peak F1) and stability across contamination levels, confirms that its scoring mechanism is better suited to capture anomalies in the complex numerical space of network flow features [16]. The high Precision achieved by both models (1.0) means few normal points are incorrectly flagged (low false alarm rate), which is important for operational trust. The selection and comparison of IF and RRCF directly addresses the resource constraints (low processing power and memory) inherent to IoT environments [3].

5. CONCLUSION

This research successfully utilized a comparative, semi-supervised framework to evaluate Isolation Forest (IF) and Robust Random Cut Forest (RRCF) for real-time intrusion detection in IoT networks, analyzing their detection

performance and Operational Efficiency (Evaluation Time) across contamination thresholds of 0.01, 0.05, and 0.10. The analysis established that both IF and RRCF are demonstrably suitable alternatives to resource-intensive centralized Deep Learning models, fulfilling the need for lightweight, edge-deployable anomaly detection. The empirical results confirmed that the Robust Random Cut Forest (RRCF) algorithm is the superior model for detection performance, achieving an F1 score of approximately 0.394 and demonstrating high F1 stability between the C=0.05 and C=0.10 thresholds. However, this performance came at a cost: RRCF was approximately 4.8 times slower (1.482 seconds latency vs. 0.306 seconds), while Isolation Forest, despite its speed, failed to provide acceptable detection capability ($F1 \approx 0.069$). Therefore, RRCF is concluded to be the preferred operational choice, as its detection reliability outweighs the operational efficiency of the statistically ineffective IF model. The core conclusion for viable lightweight NIDS implementation at the IoT edge is that maximizing F1 score stability is the decisive factor, leading to the selection of RRCF as the preferred operational choice due to its detection reliability.

Future research should focus on refining the operational robustness of the chosen algorithm. Specifically, this includes moving away from static contamination assumptions by implementing dynamic threshold adjustment techniques, which would allow the model to adapt autonomously to fluctuating anomaly rates in a live stream environment. Furthermore, testing the framework within a true streaming pipeline architecture, such as an Apache Kafka deployment, will provide a more realistic assessment of real-time performance and resource consumption. Finally, expanding the feature set to include categorical attributes (with appropriate encoding techniques) and evaluating performance against a broader, more heterogeneous IoT dataset will validate the framework's generalizability across diverse IoT use cases.

References

[1] M. Perez, "IoT Security: Risks, Challenges, and Best Practices in Securing the IoT," *Digi.com*, 2025.

[2] S. K. Sahu and K. Mazumdar, "Exploring security threats and solutions Techniques for Internet of Things (IoT): from vulnerabilities to vigilance," *Frontiers in Artificial Intelligence*, vol. 7, pp. 1397480–1397480, May 2024, doi: https://doi.org/10.3389/frai.2024.1397480.

[3] "Exploring the Impact of Outlier Variability on Anomaly Detection Evaluation Metrics," *Arxiv.org*, 2023.

[4] "What Are the Challenges of Integrating Intrusion Detection in IoT Devices? - Sentinel Over Watch," *Sentinel Over Watch*, May 16, 2024.

[5] E. Gyamfi and Anca Jurcut, "Intrusion Detection in Internet of Things Systems: A Review on Design Approaches Leveraging Multi-Access Edge Computing, Machine Learning, and Datasets," *Sensors*, vol. 22, no. 10, pp. 3744–3744, May 2022, doi: https://doi.org/10.3390/s22103744.

[6] M. Bartos, Abhiram Mullapudi, and S. Troutman, "rrcf: Implementation of the Robust Random Cut Forest algorithm for anomaly detection on streams," *The Journal of Open Source Software*, vol. 4, no. 35, pp. 1336–1336, Mar. 2019, doi: https://doi.org/10.21105/joss.01336.

[7] D. Paolini, P. Dini, Ettore Soldaini, and S. Saponara, "One-Class Anomaly Detection for Industrial Applications: A Comparative Survey and Experimental Study," *Computers*, vol. 14, no. 7, pp. 281–281, Jul. 2025, doi: https://doi.org/10.3390/computers14070281.

[8] "What are the advantages of real-time anomaly detection?," *Milvus.io*, 2025.

[9] C. Mahler, "Anomaly Detection for IoT: A Basic Primer," *IIoT World*, Jun. 20, 2024.

[10] S. Guha, N. Mishra, G. Roy, and Okke Schrijvers, "Robust random cut forest based anomaly detection on streams," *Amazon Science*, 2016.

[11] P. Spadaccino and F. Cuomo, "INTRUSION DETECTION SYSTEMS FOR IOT: OPPORTUNITIES AND CHALLENGES OFFERED BY EDGE COMPUTING AND MACHINE LEARNING."

[12] A. Si-Ahmed, M. A. Al-Garadi, and N. Boustia, "Survey of Machine Learning Based Intrusion Detection Methods for Internet of Medical Things," *arXiv.org*, 2022.

[13] M. Hana, Ibtissame Aouraghe, Oussama El Haouari, Ghizlane Khaissidi, and Mostafa Mrabti, "A Survey of Artificial Intelligence Techniques in Intrusion Detection for the Internet of Things," *Journal Européen des Systèmes Automatisés*, vol. 58, no. 6, pp. 1189–1196, Jun. 2025, doi: https://doi.org/10.18280/jesa.580609.

[14] T. Smolen and Lenka Benova, "Comparing Autoencoder and Isolation Forest in Network Anomaly Detection," pp. 276–282, May 2023, doi: https://doi.org/10.23919/fruct58615.2023.10143005.

[15] Sristi Vashisth, "Dynamic Anomaly Detection in Resource-Constrained Environments: Harnessing Robust Random Cut Forests for Resilient Cybernetic Defense," *Informatica*, vol. 48, no. 23, Dec. 2024, doi: https://doi.org/10.31449/inf.v48i23.6862.

[16] Amol Mavuduru, "How to perform anomaly detection with the Isolation Forest algorithm | Towards Data Science," *Towards Data Science*, Nov. 24, 2021.

[17] "IsolationForest," *scikit-learn*, 2025.

[18] akshara_416, "Anomaly detection using Isolation Forest - A Complete Guide," *Analytics Vidhya*, Jul. 26, 2021.

[19] A. Aldaej, T. A. Ahanger, and I. Ullah, "Deep Learning-Inspired IoT-IDS Mechanism for Edge Computing Environments," *Sensors*, vol. 23, no. 24, p. 9869, Dec. 2023, doi: https://doi.org/10.3390/s23249869.

[20] "Anomaly Detection: Isolation Forest Algorithm :: My New Hugo Site," *Github.io*, 2018.

[21] "Detect Outliers," *Mathworks.com*, 2023.

[22] "Detect Outliers and Plot Contours of Anomaly Scores," *Mathworks.com*, 2025.